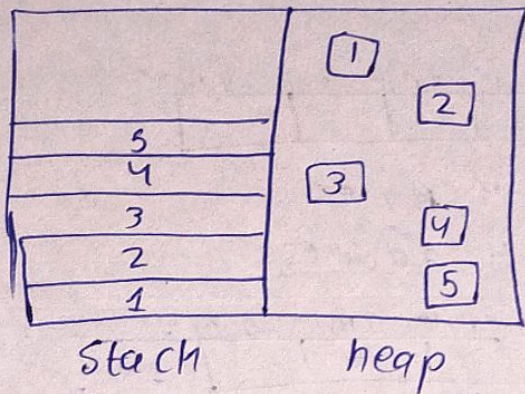


①



primitive datatypes are stored here

non-primitive datatypes are stored here

8 GB RAM :-

1 GB $\rightarrow$ 1024 MB $\rightarrow 2^{10}$ MB

1 MB $\rightarrow$ 1024 KB $\rightarrow 2^{10}$ KB

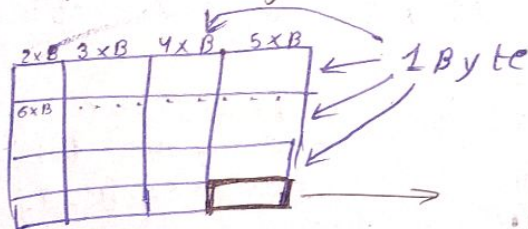
1 KB $\rightarrow$ 1024 Bytes $\rightarrow 2^{10}$ Bytes.

$8 \times 2^{10} \times 2^{10} \times 2^{10}$

$\rightarrow 8 \times 2^{30}$ Bytes are there in your phone.

Byte Address $\rightarrow$ each byte has an address.

memory is stored in bytes
& each byte has an address.



data
72
address.
6B42F

so in Computer Systems memory $\rightarrow$ each Byte has

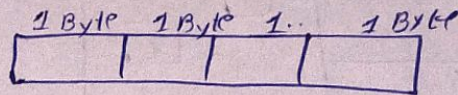
Byte Address

(Bit address nahi hota $\rightarrow$ (each Bit kay liye address) X .

word → har hitni byte ka ek address.

word = 1 Byte

har Byte ka
ek address

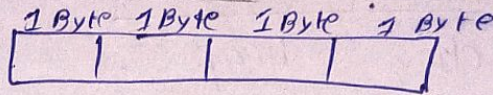


↳ 4 Bytes

↳ 4 address.

word = 2 Byte

har 2 Byte ka
ek address



↳ 4 Bytes

↳ 2 address.

So Memory

- Bit addressible ✗
(Har Bit ka address)
- Byte addressible ✓
(each Byte has its address)
- word addressible ✓
(har multiple bytes ka ek address)

• where is memory address stored?

↳ memory address is not stored. its calculated.

ex:-

	0	1	2	3	4
A	1	2	4	7	12
B	18	42	30	5	7
C					
D					
E					

So now if we want to get the value of 42 which is at address [B, 1], you can simply just access [B, 1].

So data are stored in memory,
but memory address is not stored.

(Its stored in format) & to access we just
simply access that mem add.

(2)

let a = 10;

let b = 30;

let c = a;

call by value

c = 50;

let obj1 = {

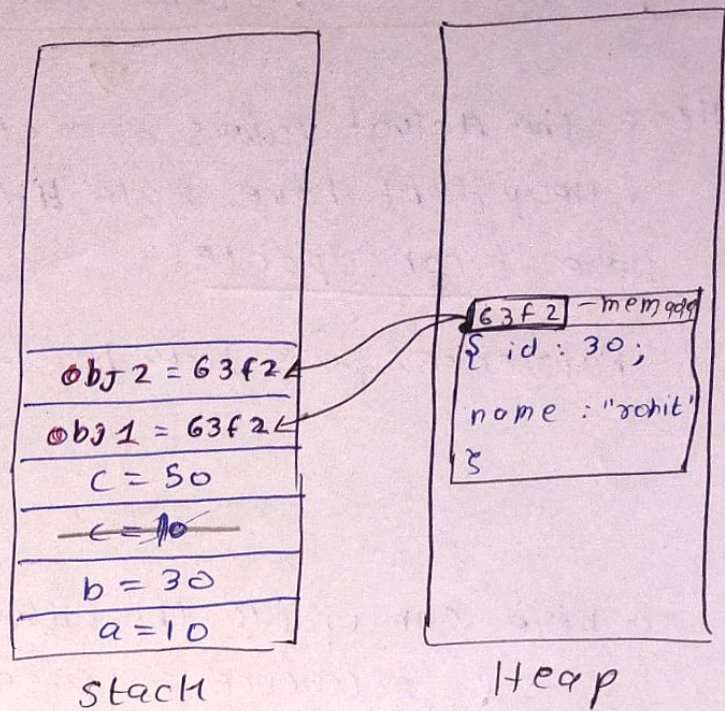
id : 20;

name : "rohit";

}

let obj2 = obj1

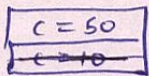
call by reference.



so everything is kept in stack, even - obj1, obj2 -
but unka data are stored in Heap. & Thus its
like this → obj1 = mem.add(63f2).

so when we do c = a then call by value hoga,
but when we do obj2 = obj1 then call by reference
hota hai & just the mem.add is assigned.

• primitive → Immutable → cannot be changed/updated.

Thus you can see that  ← yaha par update nahi
karskega.

Ⓟ The main reason is Stack Rule → cannot be changed/updated. It stores in a new location

• non-primitive → mutable → can be changed/updated



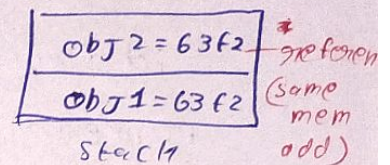
Here the actual values are stored in Heap.

& Heap don't have rule like (stack → no update)

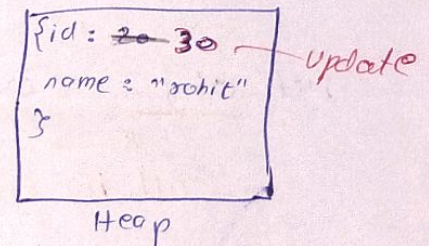
Here it can update.

& Reference pass hota hai → meaning mem. add is assigned/shared.
/ passed.
& not the actual values.

so here can update the value,
& reference is passed.



obj2.id = 30;



So, we have both seen's :-

1) stack → one by one store → Stack Rule → cannot be updated → Jo bhi datatypes stored in Here (stack)

Thus, call by value hota hai when passed.

& new location par store hota hai when updated.

So these datatypes will be immutable (→ stack Rule)

called primitive datatypes
(number, string, bigint, ...)

(3)

2) Heap → anywhere store → can be updated → Jo bhi datatypes/ values stored here (Heap)

Thus, call by reference

hota hai when passed, ~~that~~ (∴ memory bachat)

& simply updated the value there only ^(on that mem add) when updated.

so these datatypes will be mutable (∴ can update) in Heap

called non-primitive datatypes

(obj, arr, fun)

So both are Imp, - (u cannot say - Heap is better & only keep Heap - No stack.) X
both have their own purpose as per memory management.

• But why Stack Rule is there in JS, but not in C++?

cannot update there

(a = 10
a = 20)

value is passed, that mem. add.

(c = a)

↳ Since, → stack may - one by one-line say store hota hai; and if u try to update

C++ → int var = 10
var = 20

sub = 42
var = 10

Stack

In C++ we explicitly said int → 8 byte Reserve & vaha par u can update values ∴ overflow nahi hoga.

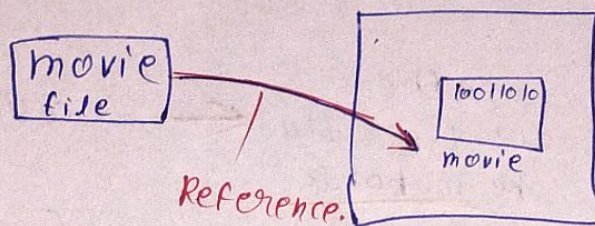
JS → let var = 10 8 byte
var = "Hello" 12 byte

sub = 42
var = 10

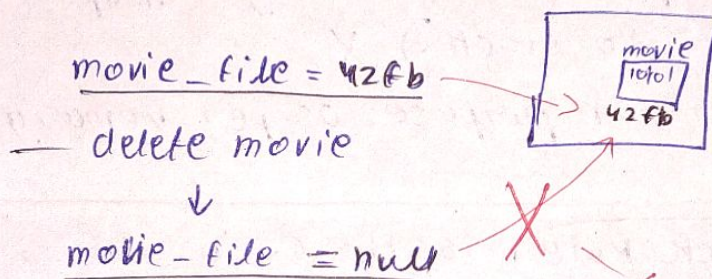
In JS dynamically typed hai (can change the datatype) now if we update here only, then abb 4 byte extra It wants but agay peechay bhara hai.

That's why → due to dynamically typed, JS may Stack Rule hai.

Extra. data Recovery :- seen.



So if u have a movie, and u delete it then the movie is not actually deleted → (all the 10011010, are converted into 0000000)
 since reference is deleted.



So the Reference is deleted and not the movie.

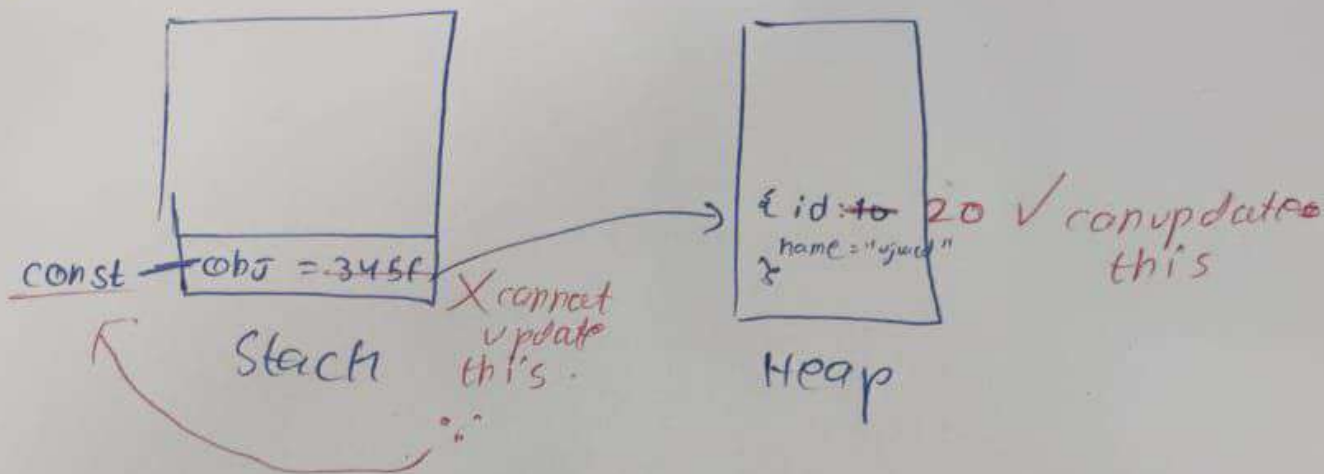
∴ same 101011 - 110 - 00000. karna may bahut time lagayaa - so it doesn't do that - what it does is
 it overrides that data when ^(store) u put something new data → (new movie, game)

so that's → when u delete the movie (4.2GB) then only reference is deleted not the movie, but in the storage if u see then -4.2GB hogaya.

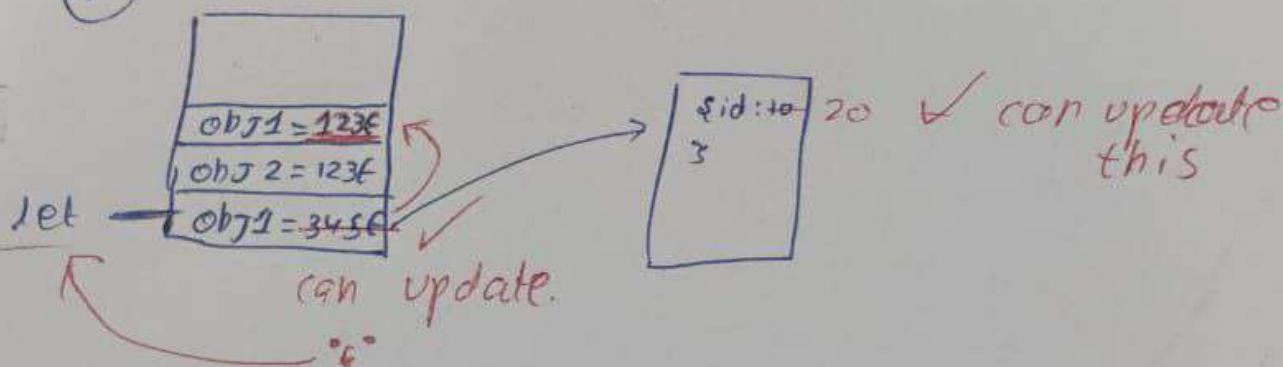
* yeah issdile hua since now that 4.2GB of memory is free to be overwritten, so u can put new movie, etc over there.
 (Samjha)

So benefit of this technique → no need to do operations ~~110101 → 0000~~
 But drawback → that data is not actually deleted so can be recovered.

②



①



& after update it
is stored in a new
location.

& previous location discarded.